Single-Key Cut-and-Choose for Keyed-Verification Blind-Signature Credentials: A Compact Construction and its Security Intuition

Abstract

A blind signature lets a user obtain a credential without revealing its message to the signer. In the BLNS23 scheme, proving that the user's request is well formed is expensive because the proof must check hash computations. Replacing this proof with cut-and-choose run in parallel lanes avoids it entirely. This document explores a compact form of that idea: the signer holds *one* key, and the user commits to all branches at once by summing them into a single aggregate, binding each branch individually by a hash. The user sends the aggregate, the seeds of the branches selected for checking, and the hashes of the unopened branches; the challenge is taken over the aggregate and the full list of branch hashes, so the branch table is fixed before the challenge exists. The signer *peels* the opened branches off the aggregate and signs the remaining sum. The first message then carries one commitment vector, κ seeds, and κ hashes: with the BLNS23 keyed-verification parameters and $\kappa = 512$ lanes, about 143 KiB. We give the construction, its correctness calculation, drawings of the exchange, and a random-oracle analysis of its security core. The design's key property is *structured-sum fixpoint hardness*: roughly, that no attacker can force the signed sum into a chosen shape without about $2^{\kappa/2}$ work. We prove this property for the challenge game in the classical random-oracle model (Theorem 5.4): an attacker making Q_h challenge-hash queries and completing Q sessions aims a peeled sum at a committed target set T, or places two accepted sessions in a committed relation T_{rel}, only with probability $O\big(Q_h\,(|T| + Q\,|T_{\mathrm{rel}}|)\,2^{-\kappa}\big)$, and every accepted session retains more than $\kappa/2$ honestly formed, challenge-selected branch vectors except with probability $Q_h\,2^{-\kappa/2}$. What remains heuristic is the step from uncontrolled responses to lattice hardness, the one-more accounting, and the quantum random-oracle extension (Section 7). Under that remaining intuition the best attacks we know are the copying-and-combining attacks of Section 5.6, costing about $2^{\kappa/2}$ classical or $2^{\kappa/4}$ quantum work, so $\kappa = 512$ is the starting point.

Security status. The challenge-game core of the construction is proven in the classical random-oracle model (Section 5.5, Theorem 5.4). The proof does not extend to quantum query access, the reduction from uncontrolled responses to a standard lattice assumption remains heuristic, and one-more unforgeability is unaccounted for (Section 7). The scheme must not be deployed as a secure credential system.

1 Introduction

In the keyed-verification blind signature of Beullens, Lyubashevsky, Nguyen, and Seiler (BLNS23) [1], the user's first message contains a zero-knowledge proof that a lattice commitment was formed correctly, including correct evaluations of hash functions. That proof dominates the cost of issuance.

Cut-and-choose offers a different route: the user prepares many candidate *branches*, reveals some of them for checking, and the signer uses only the unchecked rest. Run in κ parallel *lanes*, in

the spirit of parallel-instance cut-and-choose [2], the idea replaces the well-formedness proof with per-lane spot checks. This document explores a compact form of the construction:

- **One key.** The signer holds a single secret vector $\mathbf{s}$ and a single public matrix B. There are no lane keys.
- **One aggregate commitment, bound leaf by leaf.** The user sums all 2κ branch vectors into a single aggregate $\mathbf{c}_{\text{agg}}$ and commits to each branch individually by a *leaf hash*. The cut-and-choose challenge is derived from the aggregate together with all 2κ leaf hashes, so the branch table is committed before the challenge exists. The individual branch vectors are never transmitted: opened branches are regenerated from seeds, unopened branches are committed by their hashes only.
- **Peeling.** The signer recomputes the challenge, expands the opened branches from their seeds, subtracts them from the aggregate, and signs the remaining sum with its one key.

The appeal is concrete. With the BLNS23 keyed-verification parameters and $\kappa = 512$ lanes, the first message carries a single commitment vector, κ seeds, and κ leaf hashes: about 143 KiB, none of it a proof of hash evaluations. The signer's response proof covers a single linear key relation, and the signer stores one key.

The cost is equally concrete, and we state it up front: the BLNS23 proof route — extraction from the user's well-formedness proof — no longer exists, because cut-and-choose removed that proof, and the selected branches are never revealed in any extractable form: their leaf hashes commit without opening. No replacement reduction is known. Security here rests on the interaction of the challenge hash with the aggregate, which we call *structured-sum fixpoint hardness*; we prove its challenge-game content in the classical random-oracle model (Section 5.5), while the step from its conclusion to standard lattice hardness remains unstudied.

Roadmap. Section 2 fixes notation. Section 3 gives the construction with drawings. Section 4 shows why honest credentials verify. Section 5 gives the security intuition and analysis: the commit-before-challenge order, the fixpoint wall, the sum-to-target (Wagner) analysis, a random-oracle proof of the fixpoint game, the splicing attacks and their costs, the common-message strengthening that demotes splicing to duplication, and blindness. Section 6 estimates sizes. Section 7 lists what remains unproven.

2 Notation and basic tools

We work in the setting of the BLNS23 keyed-verification instantiation [1]. Let R_q be its ring, with dimension $d = 64$ and $\lceil \log_2 q \rceil = 152$. Vectors of $n = 93$ ring elements are written in bold, as are matrices. The rounding map $\lceil \cdot \rfloor_p : R_q \to R_p$ with $p = 4$ keeps the high bits of each coefficient. A 32-byte seed z is expanded by an extendable-output function (XOF) into pseudorandom bytes; a fixed sampling procedure turns those bytes into messages and bounded-distribution vectors. Labels fed to hash functions are distinct strings that separate the different uses.

A *lane* is an index $i \in \{0, \ldots, \kappa - 1\}$. A *branch* is one of two candidates $b \in \{0, 1\}$ prepared in a lane. A *session* is one issuance attempt, identified by a session identifier sid that names the protocol version, the issuer, and the key epoch, and carries fresh values from both parties.

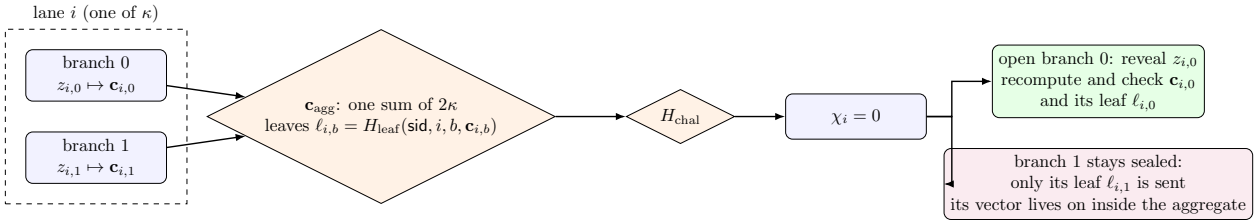

Figure 1: One lane under the aggregate commitment. Both branches enter the single sum $\mathbf{c}_{\text{agg}}$ and the leaf list; the challenge is taken over both, so the branch table is fixed before $\boldsymbol{\chi}$ exists. The challenge bit χ_i elects which branch is opened and checked. The other branch is never transmitted or checked: only its leaf hash is sent, and its vector remains inside the aggregate. Roles reverse when $\chi_i = 1$.

3 The construction

3.1 Preparing the branches

The user independently constructs two branches in each lane. It samples a 32-byte seed $z_{i,b}$ and expands it:

$$(m_{i,b}, \mathbf{r}_{i,b}, \mathbf{e}_{2,i,b}) = G(\mathsf{sid}, i, b, z_{i,b}),$$

where $m_{i,b}$ is 32 bytes and the two vectors have the prescribed bounded distributions. It computes

$$\begin{aligned}
\rho_{i,b} &= H_{\rho}(\mathsf{sid}, i, b, \mathbf{r}_{i,b}, \mathbf{e}_{2,i,b}),\\
\mathbf{u}_{i,b} &= H_{R}(\mathsf{credential\text{-}lane}, i, m_{i,b}, \rho_{i,b}),\\
\mathbf{c}_{i,b} &= B\,\mathbf{r}_{i,b} + \mathbf{e}_{2,i,b} + \mathbf{u}_{i,b},\\
\ell_{i,b} &= H_{\text{leaf}}(\mathsf{sid}, i, b, \mathbf{c}_{i,b}).
\end{aligned}$$

Three details are essential later. First, the tag $\rho_{i,b}$ binds the branch to its session: valid branches can never be reused across sessions. Second, the lane index i sits *inside* the lane hash H_R, so a branch prepared for lane i is meaningless in any other lane, even though all lanes share the same matrix B. Dropping the lane label would let an attacker move branches between lanes freely; it is mandatory here. Third, the *leaf hash* $\ell_{i,b}$ commits the branch to its position (i, b) and to the session before the challenge exists; because it covers sid, even content that is never opened cannot be replayed across sessions. Section 3.2 explains why the challenge must be taken over these leaves and over the aggregate alike.

3.2 One aggregate commitment and the challenge

The user sums all 2κ branch vectors into one aggregate

$$\mathbf{c}_{\text{agg}} = \sum_{i=0}^{\kappa-1} \big(\mathbf{c}_{i,0} + \mathbf{c}_{i,1}\big) \in R_q^n,$$

and only then derives the challenge from the aggregate and the complete leaf list

$$\boldsymbol{\chi} = (\chi_0, \ldots, \chi_{\kappa-1}) = H_{\text{chal}}(\mathsf{protocol}, \mathsf{issuer}, \mathsf{pk}, \mathsf{sid}, \mathbf{c}_{\text{agg}},\ \ell_{0,0}, \ell_{0,1}, \ldots, \ell_{\kappa-1,0}, \ell_{\kappa-1,1}) \in \{0,1\}^{\kappa}.$$

Both inputs are load-bearing, for different reasons. The leaf hashes fix the branch *table* — which vector occupies each position (i, b) — before the challenge exists. Without them the challenge would

commit only to a sum, and a sum does not determine its summands: the user could choose the decomposition of the aggregate into branches *after* seeing the challenge, and the per-lane soundness argument of Section 5.2 would not apply. The aggregate must additionally enter the challenge itself, because it is the value the signer peels: the leaves can never constrain it, since the signer never learns the unopened preimages and consistency of the aggregate with the table cannot be checked. If the challenge did not cover the aggregate, the user could fix the table, learn $\boldsymbol{\chi}$, and only then choose the aggregate so that the peeled sum equals a chosen vector — the key-recovery attack of Section 5.1 at constant cost. The encoding of the aggregate and the leaf list must be canonical and the domain separation labels fixed, so that the hash input has only one interpretation.

The first message is

$$\mathsf{User} \longrightarrow \mathsf{Signer}: \quad \mathsf{sid},\ \mathbf{c}_{\mathrm{agg}},\ \{z_{i,\chi_i}\}_{i=0}^{\kappa-1},\ \{\ell_{i,\,1-\chi_i}\}_{i=0}^{\kappa-1}.$$

Note what is *not* sent: the unopened branch vectors. The opened leaves are recomputed from the seeds; the unopened leaves arrive as hashes only. The aggregate remains the only commitment to the *values*, but the leaf list is a commitment to the *table*: every branch, including junk that is never opened, is position-bound and session-bound before the challenge exists.

3.3 Peeling and the aggregate response

The signer expands every revealed seed, reconstructs each opened branch, and computes its leaf hash. Together with the received unopened leaf hashes this reassembles the full leaf list, and the signer recomputes $\boldsymbol{\chi}$ from the received aggregate and that list. It checks that each reconstructed branch was formed for this session and lane (labels, session identifier, lattice equation, size bounds, encoding), and reconstructs the opened set $\{\mathbf{c}_{i,\chi_i}\}_i$. If any check fails it stops without returning any secret-dependent value. Otherwise it *peels* the opened branches off the aggregate:

$$\mathbf{C}_{\mathrm{sel}} = \mathbf{c}_{\mathrm{agg}} - \sum_{i=0}^{\kappa-1} \mathbf{c}_{i,\chi_i} = \sum_{i=0}^{\kappa-1} \mathbf{c}_{i,\,1-\chi_i}.$$

We call $b_i^* = 1 - \chi_i$ the *selected branch* in lane i and write $(m_i^*, \rho_i^*, \mathbf{r}_i^*, \mathbf{e}_{2,i}^*)$ for its contents. The signer never sees the selected vectors individually; it holds only their sum.

The signer derives a small deterministic error from its secret key K and the complete received input,

$$e = H_{D_e}(K, \mathsf{sid}, \mathbf{c}_{\mathrm{agg}}),$$

and answers with the aggregate response

$$h = \mathbf{s}^T \mathbf{C}_{\mathrm{sel}} + e \in R_q,$$

together with one zero-knowledge proof of the single-key response relation

$$\mathcal{R}_{\mathrm{resp}}: \quad \begin{cases} h = \mathbf{s}^T \mathbf{C}_{\mathrm{sel}} + e, \\ \mathbf{t}^T = \mathbf{s}^T B + \mathbf{e}_1^T, \\ \|\mathbf{s}\| \le \beta_s,\ \|\mathbf{e}_1\| \le \beta_e,\ \|e\| \le \beta_e. \end{cases}$$

Because there is one key, the proof covers one key relation. These are linear lattice equations and norm bounds, provable with a LaBRADOR-style proof [3], and they avoid the hash evaluations that made the BLNS23 user proof expensive. The signer releases only h and the proof: partial sums, per-lane values, failure details, and timing must not leak secret-dependent information.

3.4 Finishing and presenting the credential

After checking the response proof, the user removes the blinding terms one branch at a time and rounds:

$$v = \left\lceil h - \mathbf{t}^T \sum_{i=0}^{\kappa-1} \mathbf{r}_i^* \right\rfloor_p, \qquad \sigma = \big((m_i^*, \rho_i^*)_{i=0}^{\kappa-1},\, v\big).$$

The selected seeds stay private even at presentation: a seed would reveal the blinding vectors and let the issuer match the credential to its issuance.

At presentation, the verifier (which holds the same key) recomputes the expected value and accepts exactly when

$$v = \left\lceil \mathbf{s}^T \sum_{i=0}^{\kappa-1} H_R(\mathsf{credential\text{-}lane}, i, m_i^*, \rho_i^*) \right\rfloor_p .$$

The lane hash used at presentation must not contain the issuance session identifier; the session identifier is bound into branch formation and the challenge only. An application derives one payload from the ordered message/tag list:

$$M = H_{\text{final}}(\mathsf{issuer}, \mathsf{type}, (m_i^*, \rho_i^*)_{i=0}^{\kappa-1}).$$

4 Why honest credentials verify

The unblinding step cancels most of the noise. For an honestly generated selected branch,

$$\mathbf{s}^T \mathbf{c}_i^* - \mathbf{t}^T \mathbf{r}_i^* = \mathbf{s}^T \mathbf{u}_i^* + \big(\mathbf{s}^T \mathbf{e}_{2,i}^* - \mathbf{e}_1^T \mathbf{r}_i^*\big),$$

so

$$h - \mathbf{t}^T \sum_i \mathbf{r}_i^* = \mathbf{s}^T \sum_i \mathbf{u}_i^* + \underbrace{\sum_i \big(\mathbf{s}^T \mathbf{e}_{2,i}^* - \mathbf{e}_1^T \mathbf{r}_i^*\big) + e}_{\text{correctness error}}.$$

The verifier recomputes the main term from the presented messages and tags. Acceptance requires that the correctness error never crosses a rounding boundary. The per-branch error terms have the same shape as in BLNS23's own verification equation, but here they are summed over κ lanes: the error is a sum of $2\kappa + 1$ small terms, so its size grows with the lane count (about $\sqrt{\kappa}$ in the typical randomized model, κ in the worst case), and all terms share one secret $\mathbf{s}$ and one public-key error $\mathbf{e}_1$.

5 Security analysis

We state the attacks we know, the design rules they force, the part of the security claim that can be proven in the random-oracle model (Section 5.5), and the parts that remain heuristic.

5.1 The load-bearing order: commit before challenge

The aggregate and the branch table must be fixed before the challenge exists. To see why, suppose for a moment the opposite: the user learns $\boldsymbol{\chi}$ (or its effect on which branches are opened) before fixing $\mathbf{c}_{\text{agg}}$. Then the user can pad the aggregate with a junk term

$$J = \mathbf{w} - \sum_i \mathbf{c}_{i,\,1-\chi_i}$$

so that the peeled value equals a *chosen* vector $\mathbf{w}$. Two immediate consequences:

- **Key recovery.** With $\mathbf{w}$ the j-th unit vector, the response is $h = s_j + e$: one secret-key coordinate plus small noise. A handful of sessions per coordinate, with fresh session identifiers decorrelating the noise, recovers the entire secret key.
- **Free splicing.** With $\mathbf{w}$ chosen to cancel the honest content of a previous session and inject copied content, the copying attacks of Section 5.6 cost a constant number of hash trials instead of an exponential search.

The construction forbids this by deriving $\boldsymbol{\chi}$ from the full aggregate and the complete leaf list with a hash function modeled as a random oracle, and by including fresh signer randomness in sid. The leaf list is not optional here: if the challenge covered the leaves but not the aggregate, the aggregate itself could be chosen after $\boldsymbol{\chi}$ is known, and the same padding attack would return at constant cost (Section 3.2). In a two-message protocol the signer's fresh value must come from an authenticated session setup or a previously established context; without it, the user can precompute entire exchanges long before contact. The order carries decisive weight here, because it is the *only* barrier: with a single key, nothing algebraic separates one lane's vectors from another's (Section 5.9).

5.2 Hiding malformed branches

Call a branch *openable* if a seed regenerates it and it passes every formation check. The signer checks only opened branches, so an attacker that wants a malformed branch to affect the response must keep it unopened. Because the challenge covers the complete leaf list, the branch table is fixed before the challenge exists: for a committed table containing one openable and one malformed branch in each of k attacked lanes, a uniform challenge opens all k openable branches with probability 2^{-k}; each Fiat–Shamir trial re-randomizes the challenge, so q trials succeed with probability at most about $q\,2^{-k}$. Lemma 5.1 in Section 5.5 is the exact form of this bound.

The leaf binding is what makes this argument valid, and it repairs a real defect of the bare aggregate. A sum does not determine its summands: if the challenge were taken over $\mathbf{c}_{\mathrm{agg}}$ alone, no branch table would be committed at all, and the attacker could choose the decomposition of the aggregate into branches *after* computing the challenge — presenting whichever precomputed openable branches suit the realized $\boldsymbol{\chi}$ and treating everything else as junk. The probability experiment above would then be the wrong one: per hash query the attacker would succeed whenever *any* favorable decomposition exists, a strictly larger event whose analysis resembles a list-decoding problem rather than κ independent coin flips. The leaf hashes eliminate this freedom at a cost of 16 KiB, and because each leaf covers sid, even never-opened junk is session-bound and cannot be replayed across sessions.

The essential point stands: an attacker may place *arbitrary* junk in unopened positions — the peeled sum is never checked — but any junk must now be committed, per position and per session, before the challenge is known, and the challenge decides which honest branches remain in the sum. The next subsection explains why this makes the junk hard to aim.

5.3 Why chosen inputs seem out of reach: the fixpoint wall

The dangerous attacks above all need the peeled sum $\mathbf{C}_{\mathrm{sel}}$ to land in a chosen shape: a unit vector for key recovery, or a previous session's content for cancellation. The attacker controls the aggregate, so why can it not simply choose $\mathbf{C}_{\mathrm{sel}}$?

Because the honest branches it is forced to include interfere in a challenge-dependent way. Every lane must contain at least one openable branch (otherwise the session is rejected with probability one), and the openable branches are honestly formed, session-bound vectors whose values are fixed once the leaf list is fixed. The peeled sum is

$$\mathbf{C}_{\text{sel}} = \underbrace{\sum_{i \in H} \mathbf{c}_{i,\,1-\chi_i}}_{\text{honest, chosen by } \boldsymbol{\chi}} + \underbrace{\sum_{i \in H^c} \mathbf{w}_i}_{\text{junk, committed before } \boldsymbol{\chi}},$$

where H is the set of lanes whose selected branch is honest. To aim the sum, the attacker must choose the junk to cancel the honest part — but the honest part is a pseudorandom function of the challenge, and the challenge is a pseudorandom function of the aggregate, the junk included. The dependency is circular: choosing J fixes $\mathbf{c}_{\text{agg}}$, which fixes $\boldsymbol{\chi}$, which fixes the honest selected sum, which determines the J that would have been needed (Figure 3).

Resolving the cycle is a guess-and-check search. Guessing the full challenge pattern costs about 2^{κ} trials (or $2^{\kappa/2}$ with generic quantum search). A meet-in-the-middle variant that only asks for a chosen *difference* between two sessions' peeled sums — enough for the key-recovery attack — costs about $2^{\kappa/2}$ classical trials. We know no cheaper route. This motivates the property on which everything here rests, stated informally here and proven for the challenge game in Section 5.5:

> **Structured-sum fixpoint hardness (informal statement; formal version and proof in Section 5.5).** Consider an attacker that may prepare branch tables freely, make q random-oracle queries, and run Q issuance sessions against the construction of Section 3. Then, except with probability on the order of $q\,2^{-\kappa/2}$, every accepted session's peeled sum $\mathbf{C}_{\text{sel}}$ is *uncontrolled*: its honest content is a set of honestly formed, session-fresh, hash-derived branch vectors whose selection is decided by the challenge only after the table is committed, and no two accepted sessions' peeled sums satisfy a linear relation committed before the later session's challenge query.

Given this property, the responses $h = \mathbf{s}^T\mathbf{C}_{\text{sel}} + e$ are learning-with-errors samples whose public vectors the attacker can influence but not aim, and standard intuition applies: key recovery from uncontrolled samples is the MLWE problem, and the value $\mathbf{s}^T H_R(\cdot)$ at a fresh, session-bound input behaves like a pseudorandom function (this is the HMLWE heuristic of BLNS23, modeled on its Lemma 4.3). That second step — from uncontrolled samples to lattice hardness — is *not* covered by the random-oracle analysis and remains heuristic.

5.4 Why not a sum-to-target attack?

The most direct attempt on the fixpoint wall treats the aggregate as a sum to be solved: grind seeds to build a list of candidate vectors at every position, then search for a table whose unopened branches sum to a chosen target,

$$\sum_{i=0}^{\kappa-1} \mathbf{c}_{i,\,1-\chi_i} = \mathbf{w}, \qquad \boldsymbol{\chi} = H_{\text{chal}}(\mathsf{sid}, \mathbf{c}_{\text{agg}}, \{\ell_{i,b}\}),$$

or equivalently choose $\mathbf{c}_{\text{agg}}$ adversarially and search for opened branches summing to $\mathbf{c}_{\text{agg}} - \mathbf{w}$. Two constraints stack, and each repays careful reading.

The vector equality is exact, and the space is enormous. Exactness is forced: key recovery needs $\mathbf{C}_{\text{sel}}$ to isolate key coordinates, splicing needs previous sessions' lane values reproduced exactly because the presentation check is over exact hash images, and an approximate hit $\mathbf{w} + \boldsymbol{\delta}$ does not isolate anything, since $\mathbf{s}^T\boldsymbol{\delta}$ is secret-dependent. The target is therefore one value in a space of bit-length $N = n\,d\,\lceil \log_2 q \rceil = 93 \cdot 64 \cdot 152 \approx 904{,}704$ bits. Sum-to-target over per-lane lists is the generalized birthday problem, solved by Wagner's k-tree algorithm [4] — and its cost scales with the bit-length of the sum space, not with the security parameter: with κ lists it uses time and list size about $2^{N/(1+\lfloor \log_2 \kappa \rfloor)}$. At $\kappa = 512$ this is about $2^{90{,}470}$: the per-lane lists alone would require grinding that many seeds per lane. Even the information-theoretic minimum for a solution to exist is $\kappa\, t \geq N$ for lists of size 2^t, i.e. $t \approx 1{,}767$ bits of seed grinding per position. Exact aiming is not a 128-bit problem but a 90,000-bit one; the size of the ring — the very thing that makes the commitment vectors 113 KiB each — is what protects the construction. And the challenge constraint stacks on top: a table that solved the sum would still need $\boldsymbol{\chi}$ to select the intended complement.

Why the working attacks look different. The balanced splice (Section 5.6) and the meet-in-the-middle difference attack described above never aim at a prescribed target: they manufacture their targets from copied content, and committing a junk pair with a chosen per-lane difference costs nothing. The N-bit constraint never bites; only the challenge-hiding cost remains, $2^{\kappa/2}$ classical or $2^{\kappa/4}$ quantum. The fixpoint property is, at heart, the claim that no middle path exists — no relaxation of exact aiming that stays useful to the attacker yet becomes feasible.

Caveats. Two qualifications keep this honest. First, relaxations: if a *set* T of dangerous peeled sums is exploitable — say, all sufficiently sparse differences rather than only unit vectors — the effective attack cost divides by $|T|$, and list-merging is exactly how an attacker would harvest relaxed targets. Bounding $|T|$ rigorously is part of what a proof of key-recovery resistance must do (Section 7); the target-avoidance lemma below (Lemma 5.2) makes the dependence explicit. Second, the argument is parametric: it rests on $N \gg \kappa\,(1 + \log_2 \kappa)$. Shrinking the ring, the vector length, or the challenge length to save bandwidth moves the construction toward the regime where $2^{N/(1+\log_2 \kappa)}$ crosses below $2^{\kappa/2}$ and sum-to-target attacks become the cheapest ones.

5.5 A random-oracle proof of the fixpoint game

We now prove the precise content of the boxed statement for the challenge game, in the classical random-oracle model. The proof covers everything the challenge hash is responsible for — hiding, aiming, and committed relations between sessions. What it does not cover is stated at the end.

The game. Model G, H_ρ, H_R, H_{leaf}, and H_{chal} as independent random oracles, and assume the signer never repeats a session identifier. Call a vector $\mathbf{c} \in R_q^n$ *openable at* (sid, i, b) relative to the attacker's query transcript if the transcript contains the full chain: a seed z with $G(\mathsf{sid}, i, b, z) = (m, \mathbf{r}, \mathbf{e}_2)$ inside the norm bounds, the tag $\rho = H_\rho(\mathsf{sid}, i, b, \mathbf{r}, \mathbf{e}_2)$, the image $\mathbf{u} = H_R(\mathsf{credential\text{-}lane}, i, m, \rho)$, and the equation $\mathbf{c} = B\mathbf{r} + \mathbf{e}_2 + \mathbf{u}$. A *committed input* is a tuple $X = (\mathsf{sid}, \mathbf{c}_{\text{agg}}, \ell_{0,0}, \ell_{0,1}, \ldots, \ell_{\kappa-1,0}, \ell_{\kappa-1,1})$; write $\boldsymbol{\chi}(X) = H_{\text{chal}}(\cdots, X)$. For each position (i, b) the attacker either knows a preimage $\mathbf{c}_{i,b}$ with $H_{\text{leaf}}(\mathsf{sid}, i, b, \mathbf{c}_{i,b}) = \ell_{i,b}$ or it does not; a position is *open* if a preimage is known and openable at (sid, i, b). A lane is *free* if both positions are open, *risky* if exactly one is, and *dead* if neither is. The session is accepted exactly when the challenge position

χ_i is open in every lane and the attacker produces the corresponding seeds; the peeled sum is then

$$\mathbf{C}_{\mathrm{sel}}(X, \boldsymbol{\chi}) = \mathbf{c}_{\mathrm{agg}} - \sum_{i=0}^{\kappa-1} \mathbf{c}_{i,\chi_i},$$

well defined for acceptable $\boldsymbol{\chi}$ because the opened preimages are openable, hence unique up to random-oracle collision.

Two features of the model do all the work. First, $\boldsymbol{\chi}(X)$ is uniform in $\{0,1\}^\kappa$ and independent of everything the attacker has seen before the query at X; a session submitted without such a query is answered at signing time and is counted as a query. Second, the number of committed inputs and openable chains the attacker can produce is bounded by its query count. Write Q_h for the number of challenge-oracle queries (including submitted sessions), Q for the number of accepted sessions, and Q_R for the total number of random-oracle queries.

We condition throughout on two global events whose failure probabilities are negligible at the parameters of Section 6: (i) *oracle regularity*: no collisions occur in any random oracle, and no openable chain exists that the attacker did not query — failure probability $O(Q_R^2\, 2^{-256})$; (ii) *distinctness*: within each committed input, the values $\mathbf{C}_{\mathrm{sel}}(X, \boldsymbol{\chi})$ over acceptable $\boldsymbol{\chi}$ are distinct. A collision is an equation $\sum_{i \in S} \pm(\mathbf{u}_{i,0} - \mathbf{u}_{i,1}) =$ (lattice parts) in the H_R outputs for some nonempty $S \subseteq \{0, \ldots, \kappa-1\}$; a union bound over committed inputs, subsets S, and tuples of H_R queries gives failure probability at most $2^\kappa\, Q_R^{2\kappa+1}\, 2^{-N}$ with $N = n\, d\, \lceil \log_2 q \rceil \approx 904{,}704$, which is negligible for any $Q_R \ll 2^{N/(2\kappa)} \approx 2^{883}$.

Lemma 5.1 (commit-then-challenge soundness)**.** *For a committed input with r risky lanes and no dead lane, the session is accepted with probability 2^{-r} over $\boldsymbol{\chi}$; a dead lane makes acceptance impossible. Consequently*

$$\Pr\big[\textit{some accepted session has fewer than } \kappa/2 \textit{ free lanes}\big] \;\le\; Q_h\, 2^{-\kappa/2}.$$

Proof. Acceptance requires χ_i to name an open position in every lane, and under the uniform $\boldsymbol{\chi}$ the lanes are independent: free lanes always accept, risky lanes accept with probability $1/2$, dead lanes never. The first claim follows. An accepted session with fewer than $\kappa/2$ free lanes has at least $\kappa/2$ risky lanes, so per committed input the probability is at most $2^{-\kappa/2}$; the bound is a union over the at most Q_h committed inputs. □

Lemma 5.1 is the quantitative form of Section 5.2. Its content for what follows: outside a $Q_h\, 2^{-\kappa/2}$ exception, every accepted session has more than $\kappa/2$ free lanes, and its peeled sum can be written in the affine form

$$\mathbf{C}_{\mathrm{sel}}(X, \boldsymbol{\chi}) = \mathbf{C}_0(X) + \sum_{i \in F} \chi_i\, \boldsymbol{\Delta}_i, \qquad \boldsymbol{\Delta}_i = \mathbf{c}_{i,0} - \mathbf{c}_{i,1},$$

where F is the free-lane set, $|F| > \kappa/2$, and every $\mathbf{c}_{i,b}$ is openable — honestly formed and bound to (sid, i, b). At commit time $\boldsymbol{\chi}$ is uniform and unknown, so the peeled sum is an affine image of the challenge over more than $\kappa/2$ committed honest differences. This is the precise form of "uncontrolled": the attacker learns $\boldsymbol{\chi}$ immediately after its query, so unpredictability holds only at commit time, and changing the selection requires a new query.

Lemma 5.2 (target avoidance)**.** *Fix a committed input X and a target set $T \subseteq R_q^n$, both chosen before the challenge query at X (they may depend arbitrarily on the attacker's transcript so far). Then, under distinctness,*

$$\Pr_{\boldsymbol{\chi}}\big[\textit{accepted} \;\wedge\; \mathbf{C}_{\mathrm{sel}}(X, \boldsymbol{\chi}) \in T\big] \;\le\; |T|\, 2^{-\kappa}.$$

Proof. Acceptance is possible for at most $2^{|F|}$ challenge patterns, and by distinctness the map $\boldsymbol{\chi} \mapsto \mathbf{C}_{\mathrm{sel}}(X, \boldsymbol{\chi})$ is injective on them, so at most $|T|$ acceptable patterns land in T. The challenge is uniform over 2^{κ} patterns. □

For key recovery, T is the set of peeled sums from which key material is extractable — unit vectors, and whatever sparse or structured vectors suffice — and the lemma prices the attack at $Q_h\,|T|\,2^{-\kappa}$. Every bit of $\log_2 |T|$ comes directly off the margin, which is why bounding the dangerous target set is listed as proof work in Section 7.

Lemma 5.3 (committed-relation resistance)**.** *Let session* 1 *be accepted with peeled sum* $\mathbf{C}_1$*. For any later committed input* X_2 *and any relation set* $T_{\mathrm{rel}} \subseteq R_q^n$ *committed together with* X_2 *— after session* 1 *completes, but before the challenge query at* X_2 *— the probability that* X_2 *is accepted and* $\mathbf{C}_2 - \mathbf{C}_1 \in T_{\mathrm{rel}}$ *is at most* $|T_{\mathrm{rel}}|\,2^{-\kappa}$ *under distinctness. Union over inputs and prior sessions: at most* $Q_h\,Q\,|T_{\mathrm{rel}}|\,2^{-\kappa}$*.*

Proof. Identical to Lemma 5.2 with $T = \mathbf{C}_1 - T_{\mathrm{rel}}$: at most $|T_{\mathrm{rel}}|$ acceptable patterns for $\boldsymbol{\chi}(X_2)$ satisfy the relation, and $\boldsymbol{\chi}(X_2)$ is uniform and independent of the attacker's view at commit time, session 1 included. □

Theorem 5.4 (structured-sum fixpoint hardness, classical ROM)**.** *Let the signer enforce session-identifier uniqueness, and let* $|T|$ *and* $|T_{\mathrm{rel}}|$ *be the largest target and relation sets the attacker commits. Then, except with probability*

$$Q_h\,2^{-\kappa/2} \;+\; Q_h\,|T|\,2^{-\kappa} \;+\; Q_h\,Q\,|T_{\mathrm{rel}}|\,2^{-\kappa} \;+\; 2^{\kappa}\,Q_R^{2\kappa+1}\,2^{-N} \;+\; O(Q_R^2\,2^{-256})$$

over the random oracles, every session the attacker completes satisfies: (i) its table has more than $\kappa/2$ *free lanes, so its peeled sum is an affine image of the challenge over more than* $\kappa/2$ *honestly formed, session-fresh, hash-derived branch differences committed before the challenge existed; (ii) its peeled sum avoids every target set committed before the challenge query; (iii) its peeled sum stands in no committed relation to the peeled sum of any previously accepted session.*

Proof. Union bound over Lemmas 5.1–5.3 and the two conditioning events. □

Tightness. The bounds are matched by the attacks of the preceding subsections. Hiding k malformed branches costs 2^k trials (Lemma 5.1). Aiming at a fixed target with $|T| \approx 1$ costs 2^{κ} (Lemma 5.2). A committed difference between two sessions costs $Q_h\,Q\,2^{-\kappa} \approx 1$ at $Q_h \approx Q \approx 2^{\kappa/2}$ (Lemma 5.3) — exactly the meet-in-the-middle and balanced splice figures. The $2^{\kappa/2}$ floor is therefore not an artifact of missing analysis: it is where the proven bounds stop protecting the construction.

What is not covered. Three things. First, quantum query access: the proofs sample $\boldsymbol{\chi}$ classically at query time and take union bounds over classical query transcripts; a quantum attacker queries in superposition, and the analysis needs compressed-oracle or equivalent techniques. The $2^{\kappa/4}$ quantum floor of Section 5.6 is only a generic-search estimate. Second, the step from uncontrolled peeled sums to key-recovery resistance: Theorem 5.4 says the attacker cannot aim $\mathbf{C}_{\mathrm{sel}}$, not that $h = \mathbf{s}^T\mathbf{C}_{\mathrm{sel}} + e$ with uncontrolled $\mathbf{C}_{\mathrm{sel}}$ is safe; that step is the MLWE/HMLWE heuristic discussed above, and it is where the sizes of T and T_{rel} must be justified. Third, one-more unforgeability: the rectangle relation combines responses across sessions regardless of how uncontrolled each session is individually, and the accounting of Section 5.7 is outside the challenge game entirely.

5.6 Copying and combining responses

The rounded aggregate, together with the tags, forms a message authentication code. Write

$$F_i(x) = \mathbf{s}^T H_R(\mathsf{credential\text{-}lane}, i, x), \qquad Y(x_0, \ldots, x_{\kappa-1}) = \sum_i F_i(x_i),$$

ignoring the small correctness error. Y is a sum of per-lane values, so it satisfies a *rectangle relation*: if the same valid input $x_i = (m_i, \rho_i)$ could appear in two sessions, three issued credentials on three corners of a rectangle of inputs would combine into a fourth, unissued credential by $Y_{10} + Y_{01} - Y_{00} = Y_{11}$. The combined correctness error grows only by a constant factor.

This is why session freshness is mandatory, not optional: $\rho_{i,b}$ contains a globally unique sid, so reusing a valid input across sessions requires a hash preimage. The leaf hashes reinforce this at the branch level: since $\ell_{i,b}$ also covers sid, not even unopened junk can be carried over from one session's aggregate into another's. The enforcement must cover concurrency, rollback, and distributed replay-state failures.

Fresh sessions leave the *balanced splice*. An attacker copies old lane vectors into malformed branches (copies need no valid seeds, since unopened branches are never checked), grinds the challenge so the copies stay hidden, and thereby reproduces previous sessions' lane values inside new responses. Splitting the lanes into two halves, two grinding sessions at $2^{\kappa/2}$ each produce the second and third corners, and the fourth is free: about $2^{\kappa/2}$ classical work, or $2^{\kappa/4}$ with generic quantum search. This is the cheapest attack we know, and it sets the parameter floor: $\kappa = 512$ for a 128-bit post-quantum margin. Section 5.7 shows how to demote the splice's yield from a fresh-message forgery to a duplicate of an already-issued credential; the floor stands even so, because key recovery through the fixpoint wall independently costs about $2^{\kappa/2}$.

Two honest remarks. First, the single key makes cancellation *algebraically* easy: any lane vector can cancel any other, because all values live under one $\mathbf{s}$. But the fixpoint wall of Section 5.3 forces the cancellation term to be committed before the challenge, and the challenge-dependent honest contributions re-impose one hidden-malformed-branch bit per canceled lane — so the attack cost ends up at $2^{\kappa/2}$. Second, the random-oracle analysis of Section 5.5 shows that this figure is not an artifact of missing analysis: Lemma 5.3 stops protecting the construction exactly at $Q_h\, Q\, 2^{-\kappa} \approx 1$, that is, at $2^{\kappa/2}$ — and the splice sits exactly there. What the analysis does not give is a proof that no *use* of such a relation (the one-more accounting of Section 7) is possible below it.

5.7 Binding every lane to one message

The splicing attacks combine lane values across sessions. Their output is a credential whose lanes come from different sessions — hence, potentially, a credential for a message vector that was never issued as a whole. The strengthening in this subsection compresses what such mixing can achieve: bind every lane of a session to one hidden message M. Despite arriving late in the exposition, it should be read as part of the recommended construction, not as an option. Without it, the cheapest attack we know (Section 5.6) produces a credential for a fresh, never-issued message vector at $2^{\kappa/2}$ work; with it, the same work produces only a duplicate of a credential that was already issued. The mechanism is also a prerequisite for the one-more accounting that any proof would need (Section 7).

Use a commitment scheme that hides M, cannot be opened to two messages, and can be publicly rerandomized. The user commits $\mu_0 = \mathsf{Com}(M; \phi_0)$; each branch carries a rerandomization $\mu_{i,b} = \mathsf{Com}(M; \phi_0 + \Delta_{i,b})$ with $\Delta_{i,b}$ seed-derived; μ_0 is included in the challenge hash input; opened branches are checked to be correct rerandomizations; the lane hash becomes $H_R(\mathsf{credential\text{-}lane}, i, \mu_{i,b}, \rho_{i,b})$. At presentation the user reveals M and the selected randomness, and the verifier rebuilds every μ_i^*.

The effect: a presented credential is consistent with exactly one message. A spliced credential must then reuse an already-issued message, since its lanes can only be consistent if the spliced sessions shared M. The attack is demoted from "forge a new message" to "duplicate an existing message's credential." For applications whose authorization semantics live entirely in M, duplicates are worthless and the splicing line of attack is effectively closed; where credentials are counted, serialized, or rate-limited, duplicates remain valuable and the demotion does not suffice on its own.

What the verifier must store. The demotion has a concrete operational pay-off. Because a presented credential is consistent with exactly one message, duplicate and replay suppression collapses to one short record per presentation: M, or a hash of it. Any credential the splicing attacks can produce shares its M with an honestly issued one, so the number of distinct presentable messages stays bounded by the number of issuances, and a single-use check on M enforces exactly that bound. Without the binding the same protection is far more expensive and more fragile: a spliced credential is a *new* combination that matches no single stored credential, so the verifier would have to record every presented lane input (m_i^*, ρ_i^*) — κ values per credential — and reject any presentation that reuses a lane. The binding replaces that per-lane database with one hash per message.

M as nullifier. In the vocabulary of anonymous credentials, M plays the role of a *nullifier*: the unique public value whose reuse identifies a double presentation. Two caveats come with the terminology. First, a nullifier must be unique per *issuance*, not per payload: if two users hold credentials for the same application message (an age bound, a residency claim), a single-use check on M would make them block each other. The message should therefore carry a per-issuance serial or nonce, so that M identifies the credential instance rather than the shared attribute. Second, revealing M links all presentations of the same credential to each other and to the payload. That linkage is usually the point of a nullifier — it is what makes double presentation detectable — but it is a deliberate privacy decision, and cross-context unlinkability would need a scoped image such as $H(M, \mathsf{scope})$, with the scope bound into the lane hash so that the verifier can still recompute the lane inputs.

Three honest qualifications. First, the demotion is itself only intuition: we know of no counting argument that would justify it rigorously here, and with a single key there is no fixed untouched lane on which such an argument could anchor. Second, the binding does nothing against key recovery through the fixpoint wall, which independently costs about $2^{\kappa/2}$: the parameter floor $\kappa = 512$ stands whether or not duplicates are harmful. Third, the semantic cost is real but small. The base construction already derives a single application payload $M = H_{\text{final}}(\mathsf{issuer}, \mathsf{type}, (m_i^*, \rho_i^*)_i)$ from the ordered lane list, so applications see one message either way; what is given up is only the ability to treat individual lanes as independently meaningful messages.

5.8 Blindness intuition

The signer sees: the session identifier, one aggregate vector, the κ opened seeds with their reconstructed branches, and the κ hashes of the unopened branches. It never sees the selected seeds, the selected blinding vectors, or the final rounded value. At presentation the user reveals only messages, tags, and v; the tags ρ_i^* are one-way images of hidden branch material, and the presentation hash contains no session identifier.

The intuitive hope is that a presented credential is statistically far from any issuance view: the unblinded value differs from the signer's response by the term $\mathbf{t}^T \sum_i \mathbf{r}_i^*$, whose $\mathbf{r}_i^*$ never leave the user, and the tags are pseudorandom. Making this precise against a dishonest issuer is real work

that we do not do here: it needs a sound zero-knowledge response proof (so a cheating signer cannot bias h), an account of failed and aborted requests, key-validation rules, and a statement about how the public matrix B is generated. Note that the leaf hashes of the selected branches, though visible to the signer, are one-way images of vectors the signer never learns, and are session-bound; whether they nevertheless weaken unlinkability is part of the work just named.

5.9 What this construction gives up

Relative to BLNS23, the compactness of this design costs three things:

1. **The proof route.** The BLNS23 reduction to HMLWE exists because the user's zero-knowledge proof hands the reduction the full witness of every request. Cut-and-choose removed that proof, and the response relation gives a reduction nothing to replace it with: it cannot evaluate $\mathbf{s}^T\mathbf{C}_{\text{sel}}$ on any input that is not a fresh hash image, and the peeled sum generally is not one.

2. **Extraction handles.** The leaf hashes commit to every branch, but only the opened preimages are ever revealed: the selected branches remain hidden behind their hashes, so no forking or seed-tag technique can recover them. A proof would have to reintroduce extractable per-branch commitments with transmitted openings — giving up peeling and much of the compactness — or rely on the random-oracle analysis of Section 5.5, which covers the challenge game but not the lattice step.

3. **Defense in depth.** With one key, every lane's value lives under the same secret $\mathbf{s}$, so any lane vector can cancel any other: cross-lane cancellation is algebraically possible, and only its cost — imposed by the fixpoint wall — prevents it. The commit-before-challenge order is the single barrier; there is no second line of defense behind it.

6 Efficiency sketch

Use the BLNS23 keyed-verification values: $d = 64$, $n = 93$, $\lceil \log_2 q \rceil = 152$, $p = 4$, 32-byte messages and tags, and $\kappa = 512$ lanes as the post-quantum starting point. Ring elements are packed at 152 bits per coefficient, so one ring element occupies 1,216 bytes and one commitment vector occupies $93 \cdot 64 \cdot 152/8 = 113{,}088$ bytes.

The compactness comes from exactly two features of the design, and it is worth being explicit about both.

One aggregate commitment. Because every lane is evaluated under the same key, the signer needs only the *sum* of the selected vectors, and that sum can be recovered by subtraction: the user sends one commitment vector $\mathbf{c}_{\text{agg}}$ (113,088 bytes) and the signer peels. The unopened branch vectors are never transmitted at all. Binding the branch table costs κ additional hashes: the user sends the leaf hashes of the unopened branches (16,384 bytes) and the signer recomputes the opened half from the seeds. This is the price of the fix described in Section 3.2: without the leaf list the challenge would bind no branch table at all.

Revealed seeds instead of openings. An opened branch is fully determined by its 32-byte seed: the signer regenerates the message, the blinding vectors, the tag, the commitment vector, and the leaf hash locally and checks them. The κ opened branches therefore occupy 16,384 bytes instead of κ explicit vectors with witnesses.

Two further consequences are worth noting. The response proof covers a single-key relation — one inner product and one public-key equation — so it is small in principle, and unlike the BLNS23 user proof it contains no hash evaluations. And the signer's long-term state is one secret vector and one matrix, regardless of κ.

What does *not* shrink: the credential itself. Presentation still carries κ tags and messages plus the rounded value v, since every lane contributes an independently hidden message. The savings are entirely in issuance bandwidth, signer state, and proof complexity.

Wire component	Bytes
Aggregate commitment $\mathbf{c}_{\text{agg}}$	113,088
512 opened seeds (32 B each)	16,384
512 unopened leaf hashes (32 B each)	16,384
Session identifier	32
User $\to$ signer, fixed total	145,888
Aggregate response h	1,216
Signer $\to$ user	$1{,}216 + \lvert\pi_{\text{resp}}\rvert$
MAC $(\rho_0, \ldots, \rho_{511}, v)$	16,400
MAC plus 512 32-byte messages	32,784

Table 1: Preliminary exact wire sizes in bytes at $\kappa = 512$. The first message is about 142.5 KiB, dominated by the single aggregate vector; the credential itself is about 32 KiB. The response proof is not instantiated, so its size is excluded.

Computation shifts as follows. The user expands $2\kappa = 1{,}024$ seeds and builds 1,024 branch vectors offline. The signer's online work is κ seed expansions, κ formation checks, κ leaf-hash evaluations, κ subtractions, one inner product, and one single-key proof. Issuance latency is therefore dominated by the seeded checks, not by the signing operation.

7 What a proof would still need

The random-oracle analysis of Section 5.5 proves the challenge-game content of the fixpoint property. A full security proof of the construction would additionally need:

1. **Extraction of selected branches.** The BLNS23 reduction to HMLWE exists because the user's zero-knowledge proof hands the reduction the full witness of every request. Cut-and-choose removed that proof; peeling replaced per-branch vector transmissions by hash commitments whose selected preimages are never revealed, so partial extraction techniques (forking, seed tags) have nothing to work with. A reduction-based proof of this construction must either reintroduce individually committed branches with extractable openings — giving up the aggregate's compactness — or avoid extraction entirely.

2. **A quantum random-oracle analysis, and target-set bounds.** The proof of Theorem 5.4 samples the challenge classically at query time and takes union bounds over classical query transcripts; a quantum attacker queries in superposition, and the analysis needs compressed-oracle or equivalent techniques. Separately, the bounds scale linearly with the committed target and relation sets: a proof of key-recovery resistance must bound how many peeled sums (or differences of peeled sums) key material is actually extractable from, in the presence of the correctness noise.

3. **One-more accounting.** Even with pseudorandom responses, the rectangle relation means responses combine. A proof needs the common-message binding of Section 5.7 (or an equivalent) and a counting argument that an extra distinct message forces a fresh secret-key evaluation — and no fixed untouched lane exists in this construction on which such an argument could be anchored.

4. **Sessions and grinding hygiene.** Session identifiers must never repeat within a key epoch, including under concurrency, rollback, and replication; the signer's fresh value must be in place before the user computes the challenge.

5. **Blindness.** A dishonest-issuer privacy proof covering biased responses, failed requests, key validation, later presentation, and the leaf hashes of selected branches visible to the signer.

6. **Parameters.** A joint analysis of lattice hardness, the κ-term correctness error, rounding failure, and the $2^{\kappa/2}$ / $2^{\kappa/4}$ splice floor, together with the sum-to-target safety condition $N \gg \kappa\,(1 + \log_2 \kappa)$ of Section 5.4; $\kappa = 512$ is a starting point, not an analysis.

8 Conclusion

Cut-and-choose can replace the expensive well-formedness proof of BLNS23, and the aggregate-peeling form explored here is a compact instantiation: one signer key, one summed commitment, one leaf-hash list, one opened-seed set, and a first message of about 142.5 KiB at $\kappa = 512$ lanes. The commit-before-challenge order — over the aggregate and the branch table alike — is the single load-bearing element: it forces any attempt to aim the signed sum into a guess-and-check search whose cost we conjecture to be about $2^{\kappa/2}$, and it is all that stands between the scheme and constant-cost key recovery. For the challenge game this is now a theorem in the classical random-oracle model (Theorem 5.4), and its bounds are tight: the known attacks — the rectangle, blocked by session freshness, and the balanced splice at $2^{\kappa/2}$ classical or $2^{\kappa/4}$ quantum work, its yield demoted to credential duplication by the common-message binding — sit exactly where the bounds stop protecting the construction, while direct sum-to-target aiming is pushed far beyond reach by the size of the ring (Section 5.4). What remains open is everything past the challenge game: the step from uncontrolled responses to lattice hardness, the quantum random-oracle extension, and the one-more accounting. We present the construction as a target for analysis, not for deployment.

References

[1] W. Beullens, V. Lyubashevsky, N. Nguyen, and G. Seiler. Lattice-Based Blind Signatures: Short, Efficient, and Round-Optimal. IACR Cryptology ePrint Archive, Report 2023/077, 2023. `https://eprint.iacr.org/2023/077`.

[2] R. Chairattana-Apirom, L. Hanzlik, J. Loss, A. Lysyanskaya, and B. Wagner. Π-Cut-Choo and Friends: Compact Blind Signatures via Parallel Instance Cut-and-Choose and More. IACR Cryptology ePrint Archive, Report 2022/007, 2022. `https://eprint.iacr.org/2022/007`.

[3] W. Beullens and G. Seiler. LaBRADOR: Compact Proofs for R1CS from Module-SIS. IACR Cryptology ePrint Archive, Report 2022/1341, 2022. `https://eprint.iacr.org/2022/1341`.

[4] D. Wagner. A Generalized Birthday Problem. CRYPTO 2002, LNCS vol. 2442, pp. 288–303, 2002. `https://doi.org/10.1007/3-540-45708-9_19`.

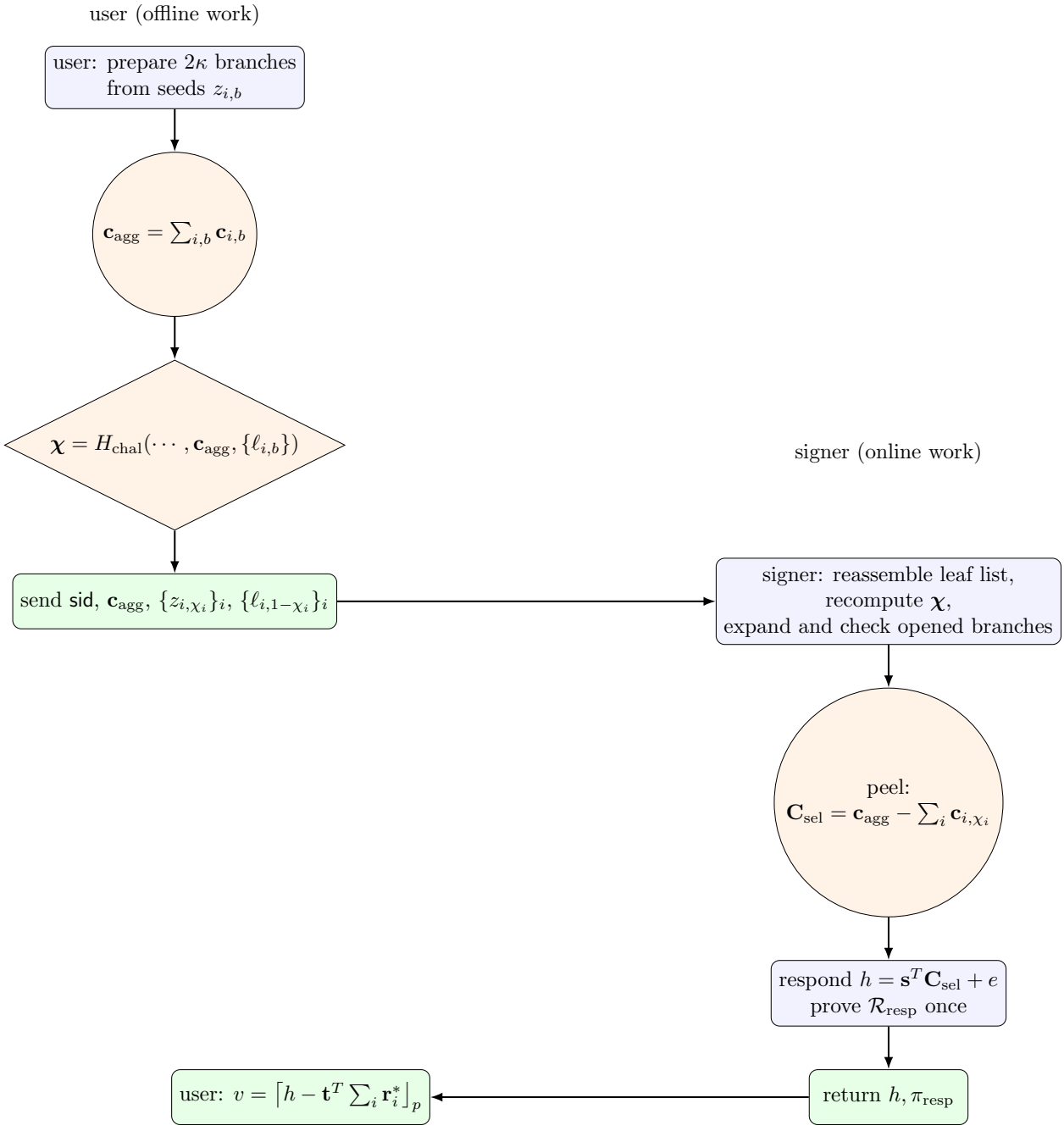

Figure 2: The two-message exchange. The user does all branch work offline and sends one aggregate vector, κ seeds, and κ leaf hashes. The signer's online work is: reassemble the leaf list and recompute the challenge, expand and check κ opened branches, one subtraction per opened branch, one inner product under its single key, and one single-key proof.

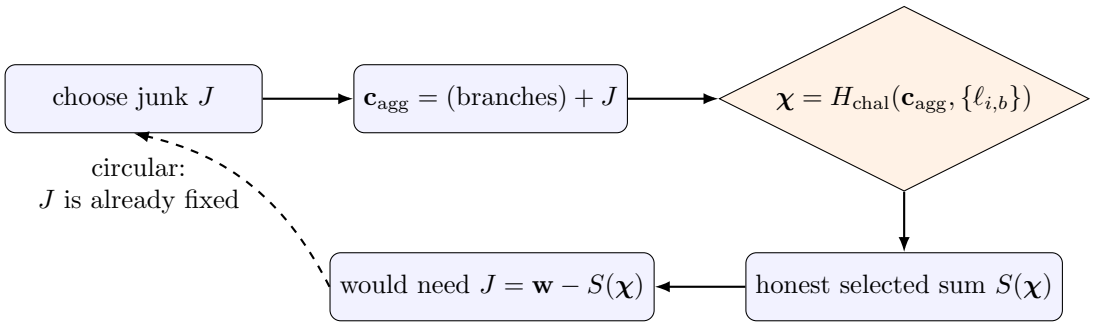

Figure 3: The fixpoint wall. Aiming the peeled sum at a target $\mathbf{w}$ requires junk that cancels a challenge-dependent honest sum; the junk is committed before the challenge exists. Resolving the cycle is a search: guess the challenge pattern, set the junk accordingly, and hope the hash returns the guessed pattern.